Software Architecture & Design Specification (SAD)

Team 9 – Smart Complaint Management System

Project Detail	Value
Project Title	Smart Complaint Management System for a Community
Group	Team 9
Course	UE24CS341A – Software Engineering
Methodology	Agile
Team Lead	Riya Saju Vithayathil (PES1UG24CS378)
Team Members	Riya Saju Vithayathil; Ritu Ranganath Kashyap; Rohan Suresh; Rohit Krishna R
Version	1.0

Revision History

Version	Date	Author	Change Summary
1.0	02-Oct-2026	Team 9	Initial consolidated SAD

1. Introduction

1.1 Purpose

This Software Architecture and Design Specification (SAD) defines the architecture and detailed software design for the Smart Complaint Management System (SCMS) for a residential community. It translates the approved requirements in the submitted SRS into an implementable software structure and provides the architectural and design basis for implementation, testing and validation.

1.2 Scope

SCMS is a centralized web application for managing complaints in a residential community. Residents can register and track complaints, while administrators and maintenance staff manage categorisation, assignment, processing, resolution, feedback and closure.

- User registration, authentication and role-based access.
- Complaint registration with category, description, location and optional attachments.
- Complaint assignment, processing, tracking, resolution and closure.
- Notifications, resident feedback and administrative reporting.

1.3 Audience

This document is intended for the project team, faculty/TAs, system users, developers, testers and future maintainers.

1.4 References

- Software Engineering Level 3 Student Submission Guide – Deliverables
- Software Requirements Specification template
- Smart Complaint Management System – SRS / Project Synopsis.

2. Document Overview

2.1 How to Use This Document

Section 3 describes the SCMS architecture, stakeholders, components, architecture pattern, technology stack, data stores, risks, requirement traceability and security architecture. Section 4 describes the detailed design, UML sequence flows, API interfaces, error handling, logging and monitoring, and UX design. Section 5 contains supporting models, glossary, references and tools.

2.2 Related Documents

The principal related documents are the Software Requirements Specification (SRS), Software Test Plan (STP), requirement traceability information and the project synopsis/proposal.

3. Architecture

3.1 Goals & Constraints

Goals:

- Support the complaint lifecycle: Submitted → Assigned → In Progress → Resolved → Closed.
- Meet NFR-001: 95% of normal requests within 10 seconds under the project test load.
- Keep modules separate and unit-testable, supporting NFR-005 and NFR-006.
- Enforce role-based access on protected requests and functions.

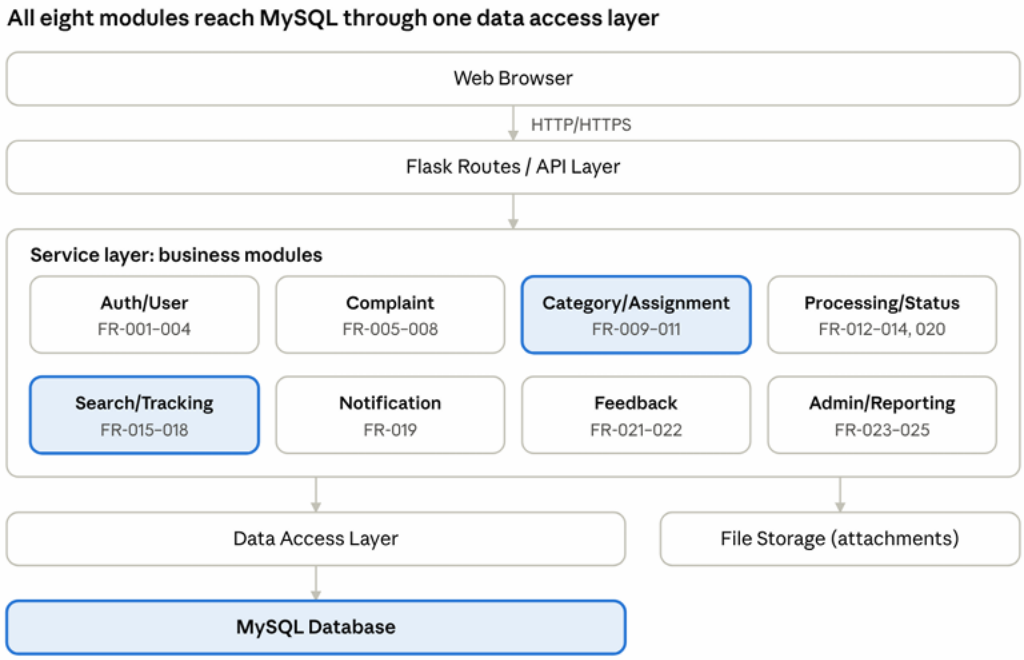
Constraints:

- Web application with Python Flask backend and MySQL relational database.
- Docker-based deployment for the planned environment.
- Initial deployment targets one residential community.
- Agile development using Git/GitHub and the course-supported engineering toolchain.

3.2 Stakeholders & Concerns

Stakeholder	Primary Concerns
Resident	Easy tracking, privacy of own complaints and accurate status.
Maintenance Staff	Clear view of assigned work and fast status/remark updates.
Administrator	Correct assignment, category control, search and monitoring.
Developers / Maintainers	Modularity, testability and consistent database schema.
Faculty / Evaluators	Traceability from requirements to architecture, design and tests.

3.3 Component (UML) Diagram



3.4 Component Descriptions

3.4.1 Auth/User Module

The Auth/User Module manages resident account creation, authentication, logout and role-based access control. It validates registration information, establishes authenticated sessions and ensures that users can access only the functions permitted for their assigned role. The module maintains user-related information and supports secure handling of authentication credentials. It is responsible for implementing FR-001 to FR-004 and supporting security requirements SEC-001 to SEC-004.

3.4.2 Complaint Module

The Complaint Module manages the creation and initial submission of complaints by authenticated residents. It validates mandatory complaint information including title, category, description and location, handles optional supporting attachments according to configured restrictions, and generates a unique Complaint ID after successful submission. The module implements FR-005 to FR-008 and supports SEC 003 and SEC-005 through input and attachment validation.

3.4.3 Category/Assignment Module

The Category/Assignment Module maintains the predefined complaint categories and lets an Administrator create, edit and deactivate them (FR-009, FR-010). It lets an Administrator assign a complaint to a department or a specific maintenance staff member, and reassign it. The first assignment moves the complaint from Submitted to Assigned (FR-011). Every assignment is recorded and a notification event is raised

3.4.4 Processing/Status Module

The Processing/Status Module manages the entire lifecycle of complaints after assignment, handling state transitions through the defined workflow sequence: Submitted -> Assigned -> In Progress -> Resolved -> Closed. It enables maintenance staff to view assigned tasks, update work statuses, attach diagnostic/resolution remarks, and record completed resolution details. Furthermore, it strictly enforces business rules for complaint closure, ensuring that a complaint cannot transition to Closed without documented resolution details and prohibiting direct closures by unauthorized actors.

3.4.5 Search/Tracking Module

The Search/Tracking Module lets a resident view the current status of their own complaints (FR-015) and exposes the status/action history kept for each complaint (FR 016). It provides search by Complaint ID, title and keyword (FR-017) and filtering by category, status, department and date (FR-018). Results are always scoped to the role of the caller.

3.4.6 Notification Module

The Notification Module creates an in-app notification for the affected users whenever an important complaint lifecycle event occurs: submission, assignment, status change, resolution and closure. Users can list their own notifications and mark them as read. A failed notification is logged and never blocks the complaint action that triggered it. The module implements FR-019.

3.4.7 Feedback Module

The Feedback Module accepts a rating and an optional comment from the resident who owns a complaint once it reaches Resolved status. It validates eligibility and stores the feedback linked to that complaint, so it can be viewed with the complaint and summarised in reports. The module implements FR-021 and FR-022.

3.4.8 Admin/Reporting Module

The Admin/Reporting Module gives the Administrator account management for residents and staff (create staff, change role or department, activate or deactivate), a dashboard of complaint counts by status, and basic reports. Accounts are deactivated rather than deleted so complaint history is preserved, and the module is read-only on complaint data. It implements FR-023 to FR-025.

3.4.9 Database (MySQL)

The Database is the single relational store used by all modules. It holds the entities described in 3.6, enforces keys and constraints, and is accessed only through the data access layer.

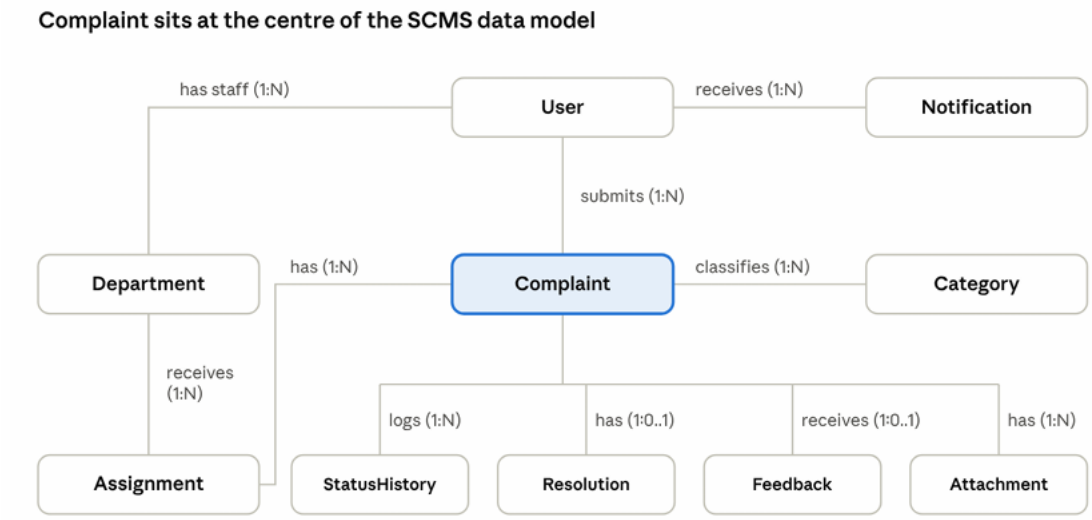
3.5 Chosen Architecture Pattern and Rationale

Pattern: Layered architecture (modular monolith): Presentation (browser) -> API/route layer -> service layer (business rules per module) -> data access layer -> MySQL. Rationale: The layers separate concerns and allow each service to be unit-tested without the full system (NFR-005, NFR-006). Role checks and the status workflow live in one place. A modular monolith deploys easily in Docker and suits a single community. Rejected alternative: Microservices were rejected because the network, deployment and consistency overhead is not justified at this scale, for a four-person team working across six sprints.

3.6 Technology Stack & Data Stores

Area	Technology / Design
Backend	Python 3 / Flask
Frontend	HTML, CSS, JavaScript
Database	MySQL
Testing	pytest
API Testing	Postman
Deployment / Containerization	Docker
Version Control	Git / GitHub
CI/CD	Jenkins
UML / Design	Draw.io / PlantUML
Security	Password hashing; HTTPS recommended in production

ER Diagram



Entity	Key Fields / Purpose
User	user_id; name; email; password_hash; contact_number; unit_address; role; department reference.
Department	department_id; name; description.
Category	category_id; name; description; is_active.
Complaint	complaint_id; resident_id; title; category_id; description; location; status; timestamps.
Attachment	attachment_id; complaint_id; file_path; file_type; file_size; uploaded_at.
Assignment	assignment_id; complaint_id; assigned_by; department_id; staff_id;

Entity	Key Fields / Purpose
	assigned_at; is_current.
Status History	history_id; complaint_id; old_status; new_status; changed_by; remarks; changed_at.
Resolution	resolution_id; complaint_id; resolved_by; details; resolved_at.
Notification	notification_id; user_id; complaint_id; message; is_read; created_at.
Feedback	feedback_id; complaint_id; resident_id; rating; comments; submitted_at.

3.7 Risks & Mitigations

Risk	Mitigation
A notification fails, so a resident misses a status change.	Failures are logged; the complaint page always shows the current status and history.

3.8 Traceability to Requirements

Requirement ID	Component	Test Case IDs
FR-001	Auth/User Module	TC-AUTH-01, TC-AUTH-02
FR-002	Auth/User Module	TC-AUTH-03, TC-AUTH-04
FR-003	Auth/User Module	TC-AUTH-05, TC-AUTH-06
FR-004	Auth/User Module	TC-AUTH-07, TC-AUTH-08
FR-005	Complaint Module	TC-CMP-01, TC-CMP-02
FR-006	Complaint Module	TC-CMP-03, TC-CMP-04
FR-007	Complaint Module	TC-CMP-05, TC-CMP-06
FR-008	Complaint Module	TC-CMP-07, TC-CMP-08
FR-009	Category/Assignment Module	TC-ASG-01, TC-ASG-02
FR-010	Category/Assignment Module	TC-ASG-03, TC-ASG-04, TC-ASG-05
FR-011	Category/Assignment Module	TC-ASG-06, TC-ASG-07, TC-ASG-08
FR-012	Processing/Status Module	TC-PRC-01, TC-PRC-02
FR-013	Processing/Status Module	TC-PRC-03, TC-PRC-04
FR-014	Processing/Status Module	TC-PRC-05, TC-PRC-06
FR-015	Search/Tracking Module	TC-TRK-01, TC-TRK-02
FR-016	Search/Tracking Module	TC-TRK-03, TC-TRK-04
FR-017	Search/Tracking Module	TC-TRK-05, TC-TRK-06, TC-TRK-09
FR-018	Search/Tracking Module	TC-TRK-07, TC-TRK-08
FR-019	Notification Module	TC-NTF-01, TC-NTF-02
FR-020	Processing/Status Module	TC-PRC-07, TC-PRC-08
FR-021	Feedback Module	TC-FBK-01, TC-FBK-02
FR-022	Feedback Module	TC-FBK-03, TC-FBK-04
FR-023	Admin/Reporting Module	TC-ADM-01, TC-ADM-02
FR-024	Admin/Reporting Module	TC-ADM-03, TC-ADM-04
FR-025	Admin/Reporting Module	TC-ADM-05, TC-ADM-06
NFR-001	Application Infrastructure	TC-PERF-01, TC-PERF-02
NFR-002	Processing/Status Module	TC-NFR-01
NFR-003	Processing/Status Module	TC-NFR-02
NFR-004	Processing/Status Module	TC-NFR-03
NFR-005	Application Architecture	TC-NFR-04
NFR-006	Application Architecture	TC-NFR-05

Requirement ID	Component	Test Case IDs
SEC-001	Auth/User Module	TC-SEC-01, TC-SEC-02
SEC-002	Auth/User Module	TC-SEC-03, TC-SEC-04
SEC-003	Auth/User + Complaint Module	TC-SEC-05, TC-SEC-06
SEC-004	Auth/User Module	TC-SEC-07, TC-SEC-08
SEC-005	Complaint Module	TC-SEC-09, TC-SEC-10

3.9 Security Architecture

The Smart Complaint Management System applies role-based access control, secure credential handling, input validation, session protection and attachment restrictions to protect user and complaint information. Security controls are designed around the major threat categories represented by the STRIDE model.

STRIDE Threat	SCMS Threat	Security Control
Spoofing	Unauthorized user attempts to authenticate as another resident.	Authentication using registered credentials and secure password handling.
Tampering	User attempts to modify requests or submit manipulated complaint data.	Server-side input validation and authorization checks.
Repudiation	User denies performing an authenticated operation.	Authenticated sessions and recording of complaint/status actions.
Information Disclosure	Unauthorized access to resident or complaint information.	Role-based access control and protected authenticated endpoints.
Denial of Service	Excessive or malformed requests affecting application availability.	Input validation, request restrictions and appropriate error handling.
Elevation of Privilege	Resident attempts to access staff/admin functionality.	Role-based authorization enforced on protected operations.

Security Requirement Mapping

- SEC-001: Passwords shall not be stored in plaintext.
- SEC-002: Residents should only access resident features, not staff or administrator features.
- SEC-003: The system should check and clean user input before using it.
- SEC-004: Users with invalid or expired login sessions should not be allowed to access protected pages.
- SEC-005: Complaint attachments shall be restricted by configured file type and size.

Security boundary: Resident → Authentication / Authorization → Application API → Business Modules → MySQL Database.

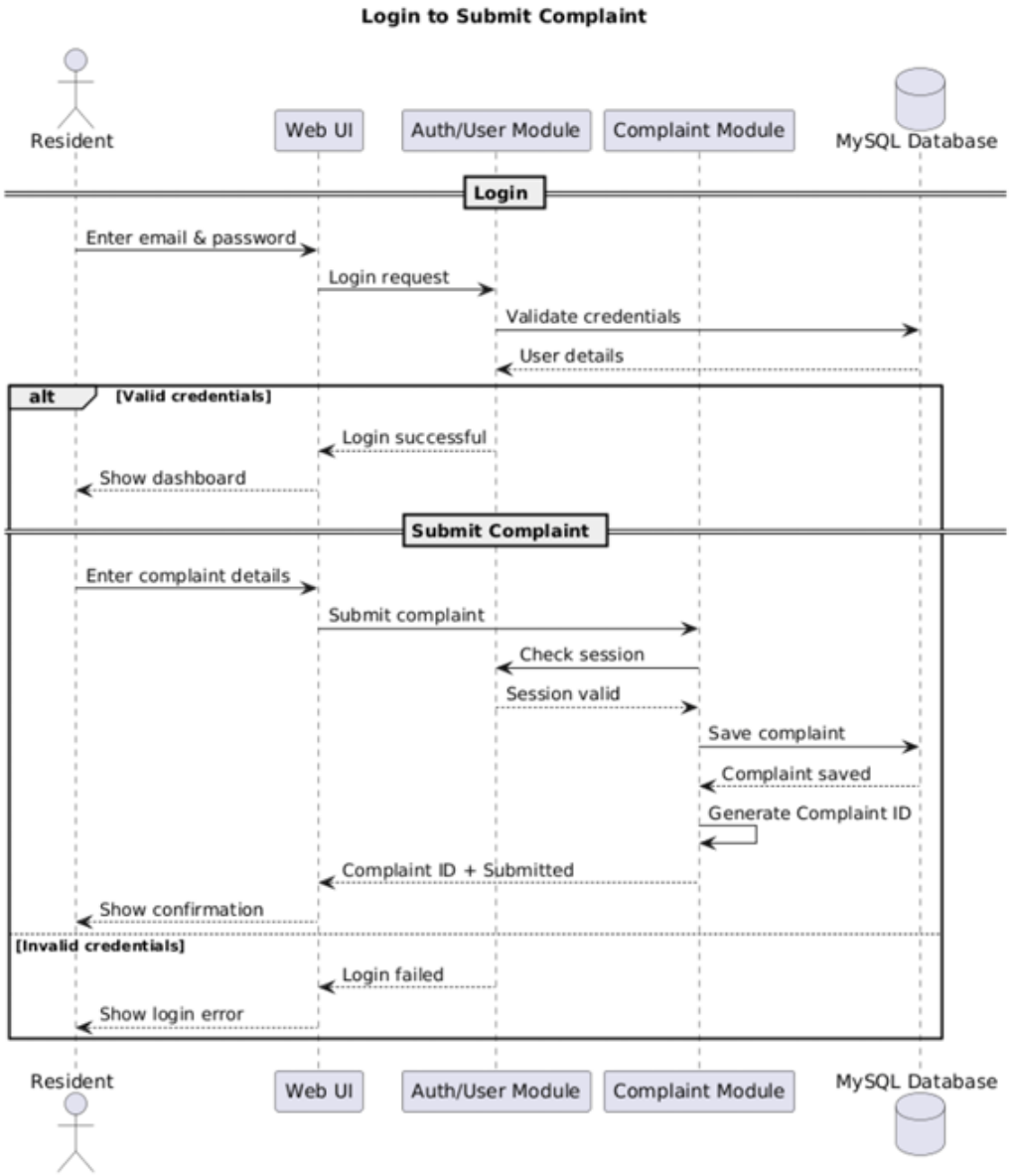
4. Design

4.1 Design Overview

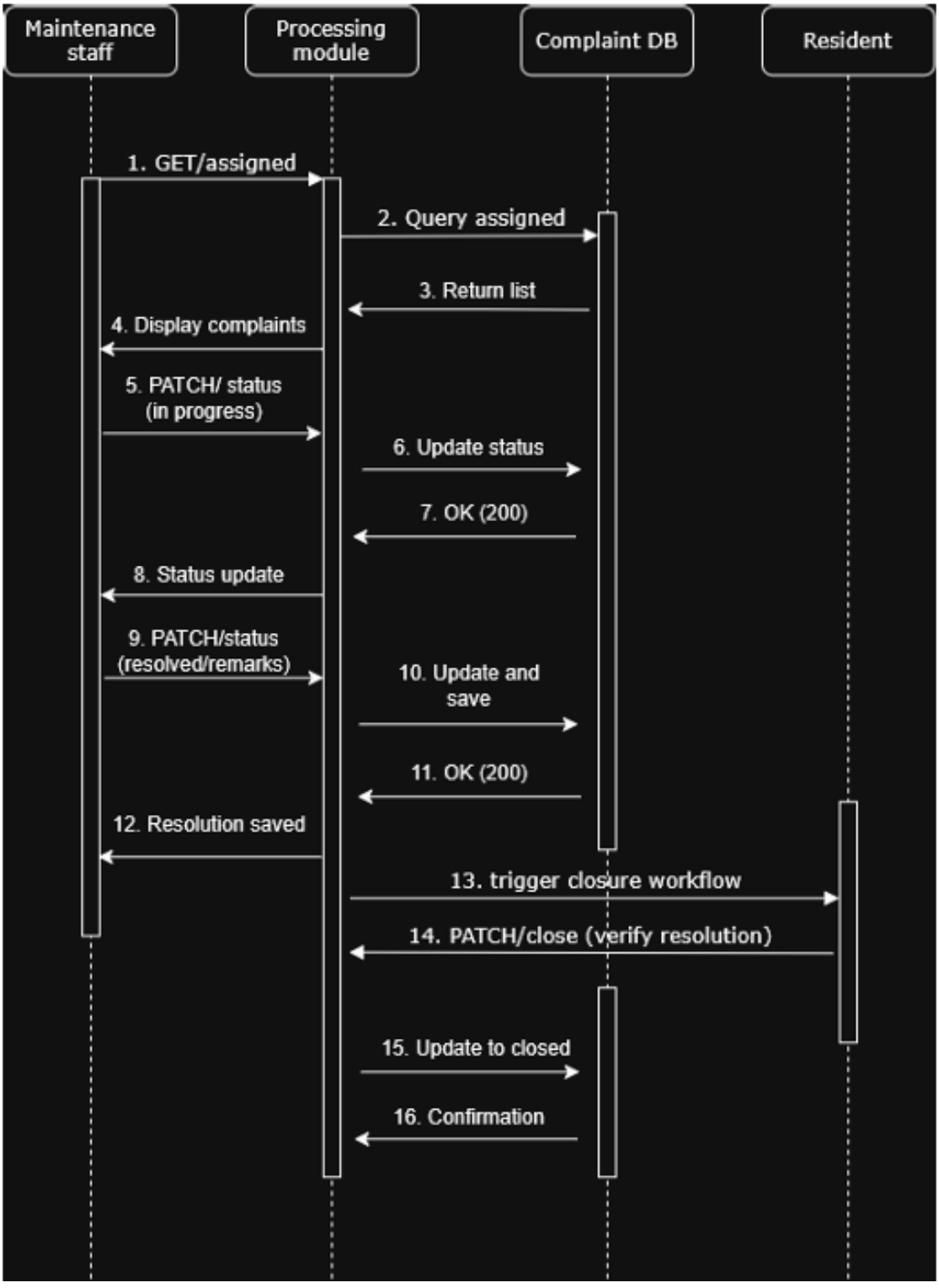
SCMS is a layered, modular Flask application. Each module has routes, a service class holding its business rules, and repository functions that talk to MySQL. A shared authentication layer attaches the user's identity and role to each request. Role checks happen in the service layer, so rules such as "residents see only their own complaints" and "only Administrators assign" are enforced regardless of the UI. All status changes and assignments write to StatusHistory and Assignment, which preserves accountability.

4.2 UML Sequence Diagrams

4.2.1 Login → Submit Complaint



4.2.2 Sequence Diagram: Process Resolve Close Workflow



4.3 API Design

The following are proposed design-level API interfaces for the system components. They define the expected method, request, response and error behavior while preserving the responsibilities and authorization rules specified by the SRS.

4.3.1 Auth/User Module

Endpoint	Method	Purpose
/api/auth/register	POST	Create resident account
/api/auth/login	POST	Authenticate user
/api/auth/logout	POST	End authenticated session
/api/users/me	GET	Retrieve authenticated user's profile

POST /api/auth/register

Request: name, email, password, contact_number, unit_address

Response: Account-created message and generated user_id.

Errors: 400 – missing/invalid required information; 409 – email/account already exists.

POST /api/auth/login

Request: email, password

Response: Login-success message and authenticated session.

Errors: 400 – invalid request; 401 – invalid credentials.

POST /api/auth/logout

Request: authenticated session/token

Response: Logout-success message.

Errors: 401 – invalid or expired session.

GET /api/users/me

Request: Authenticated session/token

Response: User profile containing user_id, name, email, contact_number, unit_address and role.

Errors: 401 – invalid or expired session.

4.3.2 Complaint Module

Endpoint	Method	Purpose
/api/complaints	POST	Submit a new complaint
/api/complaints/{complaint_id}	GET	Retrieve complaint details
/api/complaints/{complaint_id}/attachments	POST	Upload supporting attachment

POST /api/complaints

Request: title, category_id, description, location

Response: Successful submission message, unique complaint_id and status=Submitted.

Errors: 400 – missing/invalid complaint information; 401 – user not authenticated; 422 – invalid input.

POST /api/complaints/{complaint_id}/attachments

Request: multipart file upload

Response: Attachment-uploaded message.

Errors: 400 – invalid file; 401 – user not authenticated; 413 – file exceeds configured size; 415 – unsupported file type.

GET /api/complaints/{complaint_id}

Request: complaint_id path parameter and authenticated session/token

Response: Complaint details including complaint_id, title, category, description, location, status, created_at and attachment reference if applicable.

Errors: 401 – user not authenticated; 403 – user not permitted to access the complaint; 404 – complaint not found.

4.3.3 Category/Assignment and Search/Tracking Modules

Endpoint	Method	Role	Request	Success Response	Errors
/api/categories	GET	Any	none	200: category_id, name, description	401
/api/categories	POST	Admin	name, description	201: category_id, name	400, 403, 409 duplicate name
/api/categories/{id}	PUT	Admin	name, description, is_active	200: updated category	400, 403, 404, 409
/api/categories/{id}	DELETE	Admin	none	204	403, 404, 409 category in use
/api/departments	GET	Admin	none	200: department_id, name	403
/api/departments/{id}/staff	GET	Admin	none	200: user_id, name	403, 404
/api/complaints/{id}/assignment	POST	Admin	department_id, staff_id (optional)	200: complaint_id, status, department_id, staff_id, assigned_at	400, 403, 404, 409 if complaint Resolved/Closed
/api/complaints/{id}/status	GET	Resident (own), Staff (assigned), Admin	none	200: complaint_id, status, updated_at	403, 404
/api/complaints/{id}/history	GET	Resident (own), Staff (assigned), Admin	none	200: status/action history	403, 404
/api/complaints	GET	Role-scoped	q, category_id, status, department_i	200: items, page, total	400 invalid filter value

Endpoint	Method	Role	Request	Success Response	Errors
			d, from, to, page		

4.3.4 Processing/Status Module

Endpoint	Method	Request	Response	Errors
/api/v1/staff/complaints/assigned	GET	{status}	Assigned complaint list with complaint_id, title, category, location, status and created_at.	401 Unauthorized, 403 Forbidden
/api/v1/staff/complaints/{complaint_id}/status	PATCH	{status, remarks, resolution_details}	Updated complaint status and timestamp.	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
/api/v1/complaints/{complaint_id}/close	PATCH	{confirmation, closure_notes}	Closed complaint and closed_at.	400 Bad Request, 403 Forbidden

4.3.5 Notification Module

Interface / Endpoint	Method	Request	Response	Errors
notify()	Internal	event, complaint_id, recipient_ids	Notification record per recipient.	Failure is logged; calling action is not rolled back.
/api/notifications	GET	Authenticated session; optional unread=true	User's notifications, newest first.	401 – invalid or expired session.
/api/notifications/{notification_id}/read	PATCH	notification_id + authenticated session	Notification marked as read.	401, 403, 404.

4.3.6 Feedback Module

Endpoint	Method	Request	Response	Errors
/api/complaints/{complaint_id}/feedback	POST	rating (1–5), comment (optional)	feedback_id, complaint_id, rating and comment.	400 invalid rating; 401 unauthenticated; 403 not complaint owner; 404 complaint not found; 409 not Resolved or feedback already submitted.
/api/complaints/{complaint_id}/feedback	GET	complaint_id + authenticated session	feedback_id, complaint_id, rating, comment and created_at.	401, 403, 404.

4.3.7 Admin/Reporting Module

Endpoint	Method	Request	Response	Errors
/api/admin/users	GET	Optional role and active filters	User list; passwords never returned.	401 unauthenticated; 403 not Administrator.
/api/admin/users	POST	name, email, password, contact_number, role, department_id (staff only)	Account-created message and user_id.	400 invalid; 403 not Administrator; 409 email exists.
/api/admin/users/{user_id}	PATCH	role, department_id and/or is_active	Updated user record.	400 invalid; 403 not Administrator; 404 not found; 409 self-deactivation.
/api/admin/dashboard	GET	Authenticated Administrator session	Complaint counts by status, pending count and average rating.	401; 403.
/api/admin/reports	GET	type; optional from/to dates	Report type and rows of label/count.	400 invalid type/date range; 403 not Administrator.

4.4 Error Handling, Logging & Monitoring

Standardized Error Handling

Application errors use structured error information. Example:

- error_code: INVALID_STATE_TRANSITION
- message: Cannot transition complaint state from Submitted directly to Resolved.
- timestamp: 2026-10-02T20:50:00Z

Logging Standards

Structured logging is planned in JSON format. Sensitive data, including passwords, session tokens and personal or other protected information, is sanitized before logging.

Monitoring Metrics

Application metrics cover API latency, transition failure counts and overall system availability. The design targets p95/p99 latency below 10 seconds under the project load.

4.5 UX Design

Screen / Function	User	Design
Notifications	All roles	Bell icon with unread count; list newest first and link notifications to the relevant complaint.
Feedback form	Resident	Shown once the complaint is Resolved; 1–5 stars and optional comment; read-only after submission.
Admin dashboard	Administrator	Count cards per status, total, pending count and average rating.
User management	Administrator	User table with role filter; Add Staff, Edit, Activate/Deactivate. No delete.
Reports	Administrator	Report type and date-range selectors with results table.

4.6 Open Issues & Next Steps

- Notification delivery remains implementation-defined; the design does not assume SMS/WhatsApp.
- The project will be developed incrementally using Agile methodology.
- Integration testing, system validation, bug fixing and final deployment remain part of the planned work.

5. Appendices

5.1 Glossary

Term	Meaning
SCMS	Smart Complaint Management System.

Term	Meaning
SAD	Software Architecture and Design Specification.
SRS	Software Requirements Specification.
STP	Software Test Plan.
RBAC	Role-Based Access Control.
Complaint ID	Unique identifier assigned to a successfully submitted complaint.
Resident / Complainant	Person who submits and tracks a complaint.
Maintenance Staff / Officer	Authorized person who processes assigned complaints.
Administrator	Authorized person who manages the system and administrative functions.
CRUD	Create, Read, Update and Delete.

5.2 References

1. Smart Complaint Management System – Team 9 Software Requirements Specification (SRS).
2. Smart Complaint Management System – Team 9 Software Test Plan (STP).
3. PES University CSE Software Engineering – Level 3 Mini-Project Guidelines and SAD Template.
4. Project Synopsis and approved requirement specifications.

5.3 Tools

The following tools were used or planned for the modelling, development and validation of SCMS:

- **PlantUML / Draw.io** – UML and architectural diagrams.
- **Git / GitHub** – Version control and collaborative development.
- **Postman** – API testing and validation.
- **pytest** – Automated testing.
- **Jira / GitHub Projects** – Agile task and issue tracking.
- **Docker** – Application containerisation.